Writely: Multi-Input Syntactical Analyzer

High-Density Architectural Walkthrough & Code Deconstruction Pipeline

Focus Areas:

- Clean Architecture Boundaries & Module Seams
- End-to-End Pipeline: Lexing → AST → Universal Checking → Orchestration → GUI
- Polymorphic Language Adapters & Multi-Language Contrast
- Architectural Tradeoffs & Resilience Strategies

Slide 2: Architectural Boundaries & Module Seams

Writely follows **Clean / Ports-and-Adapters** architectural principles:

```
graph TD
    subgraph UI ["app / GUI (JavaFX)"]
        WC["WorkspaceController"]
        App["App Bootstrap"]
    end

    subgraph Core ["core (Domain Logic & Pipeline)"]
        AO["AnalysisOrchestrator"]
        SIS["SourceIngestionService"]
        SC["SyntaxChecker Suite"]
        DA["DiagnosticAggregator"]
    end

    subgraph Adapters ["adapter (Language Plugins)"]
        PAR["ParserAdapterRegistry"]
        JA["JavaAdapter"]
        PA["PythonAdapter"]
        CA["CppAdapter"]
    end

    subgraph Domain ["domain (Pure POJOs / Records)"]
        SP["SourcePayload"]
        TK["Token / SourceSpan"]
        SN["SyntaxNode (AST)"]
        DG["Diagnostic / Severity"]
        AR["AnalysisResult"]
    end

    WC --> AO
    AO --> SIS
    AO --> PAR
    PAR --> JA & PA & CA
    AO --> SC
```

Slide 3: The Universal Seam — `ParserAdapter`

The bridge between language-agnostic orchestration and language-specific grammars:

```
public interface ParserAdapter {
    Language language();
    List<Token> tokenize(SourcePayload payload);
    ParseResult parse(SourcePayload payload, List<Token> tokens);
    RuleCapabilities ruleCapabilities();
}
```

- **Contract Isolation**: The engine never branches on language types (`if (lang == JAVA)`).
- **Decoupled AST Output**: `ParseResult` emits `Optional<SyntaxNode>` and internal parser errors.
- **Declarative Capabilities (`RuleCapabilities`)**: Each adapter declares which universal rule categories it supports (e.g., Python opts out of

Slide 4: Phase 1 — Ingestion & Source Normalization

Raw inputs enter through `SourceIngestionService` before touching any lexer.

- **Three Input Modalities**:
  i. `Single Line`: Immediate single-statement inspection.
  ii. `Code Snippet`: Unsaved multiline scratchpad buffers.
  iii. `File Upload`: Disk-backed files via file picker or tree explorer.
- **Normalization Responsibilities**:
  - Enforces UTF-8 character encoding.
  - Normalizes heterogeneous line separators (`\r\n` / `\r` → `\n`).
  - Emits an immutable `SourcePayload(content, language, inputMode, origin)`.
- **Zero-Leakage Seam**: Downstream tokenizers receive a guaranteed clean in-memory string with reliable line/offset mathematics.

Slide 5: Phase 1 — Lexical Analysis & Token Channels

Tokenizers implement `Tokenizer` and break characters into atomic lexical tokens:

```
Source Code:   "int count = 42; // counter"
Emitted Stream: [KEYWORD: int], [IDENT: count], [OP: =], [INT_LIT: 42], [SEMI: ;], [COMMENT]
```

- **Trivia Channel Architecture**:
  - **Default Channel**: Significant program tokens (keywords, literals, identifiers, operators).
  - **Trivia Channel**: Comments, whitespace, indentation markers (`isTrivia() == true`).
- **Preserving Concrete Details**:
  - Parsers ignore trivia to keep ASTs concise.
  - Syntax checkers and editor syntax highlighters retain trivia to inspect comments, strings, and whitespace formatting without losing source

Slide 6: Multi-Language Tokenizer Contrast

Language semantics dictate unique lexical tokenization strategies:

Feature	Java (`JavaTokenizer`)	Python (`PythonTokenizer`)	C++ (`CppTokenizer`)
Delimiters	Braces `{ }` emit blocks	Indentation stack emits `INDENT` / `DEDENT`	Braces `{ }` emit blocks
Line Ends	Newlines treated as whitespace	Newlines emit significant `NEWLINE` token	Newlines treated as whitespace
Directives	Annotations (`@Override`)	Decorators (`@property`)	Preprocessor `#include`, `#define`
String Types	Standard & text blocks `"""`	Single `'`, double `"`, triple `'''`, `f""`	Raw strings `R"(...)"`, wide

Slide 7: Phase 2 — Syntactic Parsing & AST Construction

Parsing transforms flat linear token streams into hierarchical syntax trees:

- **Separation of Concerns**:
  - `adapter.tokenize()` produces `List<Token>`.
  - `adapter.parse(payload, tokens)` constructs `Optional<SyntaxNode>`.
- **Two-Stage Error Handling**:
  - `ParseResult` captures structural syntax tree root.
  - Emits **Internal Parser Diagnostics** (e.g., `ERR_JAVA_MISSING_SEMICOLON`, `CPP_UNCLOSED_DELIMITER`) to facilitate parser error recovery without crashing the analysis run.
- **Fault-Tolerant Parse Trees**: Even if a syntax error occurs, the parser recovers and returns partial subtrees so the GUI still renders valid sections.

Slide 8: AST Construction: `JavaAstMapper`

Deep-dive into Java AST translation:

```
[CompilationUnit]
  └── [ClassDeclaration: "Calculator"]
        └── [MethodDeclaration: "add"]
              ├── [Parameters: (int a, int b)]
              └── [Block]
                    └── [ReturnStatement]
                          └── [BinaryExpression: a + b]
```

- **50+ Typed AstNodes**: `JavaClassDeclaration`, `JavaMethodDeclaration`, `JavaIfStatement`, etc.
- **Recursive Tree Mapping**: Flat tokens are transformed into nested immutable `JavaAstNode` instances, then mapped to universal `SyntaxNode(type, label, span, children)`.
- **Span Preservation**: Every single AST node retains exact `SourceSpan(start,`

Slide 9: Multi-Language AST Structural Contrast

How language paradigms shape their corresponding AST hierarchies:

- **Java (`JavaCompilationUnit`)**:
  - Strict nominal hierarchy: Package → Imports → Classes/Interfaces/Records → Methods/Fields → Statements.
  - Strongly typed nodes with explicit parameter, variable, and return type nodes.
- **Python (`PythonModule`)**:
  - Script/module level statements permitted without enclosing class.
  - Indentation-driven block containers (`SuiteNode`) holding statements.
  - Dynamic assignments without type specifiers.
- **C++ (`CppTranslationUnit`)**:
  - Preprocessor directives (`CppInclude`, `CppPreprocessorDirective`) sit alongside declarations.

Slide 10: Phase 3 — Universal Rule Checking Suite

In addition to language parser errors, Writely executes 6 modular `SyntaxChecker`s:

1. **`DelimiterMatchingChecker`**: Stack-based validation of `()`, `[]`, `{}` pairs across tokens.
2. **`StringAndCharacterLiteralChecker`**: Unterminated quotes, illegal escape sequences.
3. **`StatementTerminatorAndBlockChecker`**: Missing `;` in Java/C++; improper indentation in Python.
4. **`OperatorSyntaxChecker`**: Illegal adjacent binary operators (`a ++* b`), missing operands.
5. **`ControlStructureHeaderChecker`**: Malformed `if (...)`, `for (...)`, `while (...)` condition headers.
6. **`IdentifierSyntaxChecker`**: Naming convention violations, invalid start characters.

Slide 11: Phase 4 — Analysis Orchestrator Lifecycle

```
sequenceDiagram
    autonumber
    actor UI as WorkspaceController
    participant AO as AnalysisOrchestrator
    participant REG as ParserAdapterRegistry
    participant ADP as Java/Python/CppAdapter
    participant SC as SyntaxCheckers
    participant DA as DiagnosticAggregator

    UI->>AO: analyze(SourcePayload)
    AO->>REG: resolveRequiredAdapter(payload)
    REG-->>AO: ParserAdapter instance
    AO->>ADP: tokenize(payload)
    ADP-->>AO: List<Token>
    AO->>ADP: parse(payload, tokens)
    ADP-->>AO: ParseResult (AST + Internal Diags)
    AO->>SC: executeSupportedCheckers(tokens, AST)
    SC-->>AO: Canonical Diagnostics
    AO->>AO: Apply ADR 0015 Canonical Supersession
    AO->>DA: aggregate(mergedDiagnostics)
    DA-->>AO: DiagnosticSummary & Sorted Findings
    AO-->>UI: AnalysisResult
```

Slide 12: Phase 4 — ADR 0015 Canonical Supersession

The Problem: If a line is missing a semicolon, both `JavaParser` (internal) and `StatementTerminatorAndBlockChecker` (canonical) will flag it. Emitting both creates confusing UI duplicates!

```
Parser Diagnostic:    [JAVA_MISSING_SEMICOLON] "Syntax error on token, ';' expected"
Canonical Diagnostic: [ERR_MISSING_SEMICOLON]  "Missing statement terminator ';'" -> Fix: "Add ';'"
```

The Resolution (ADR 0015):

- When an internal parser diagnostic and a canonical diagnostic share the same `CheckCategory` and `SourceLocation` **(line, column)**:
  - **Canonical supersedes Internal**.
  - The internal diagnostic is dropped; the canonical diagnostic with its actionable `suggestedFix` is retained.
- Ensures 100% deduplicated, clean diagnostic lists for the user.

Slide 13: Phase 5 — GUI Decoupling & Reactive Rendering

How `WorkspaceController` processes `AnalysisResult` without blocking the JavaFX UI:

- **Background Worker Thread**:
  - Analysis runs inside JavaFX `Task<AnalysisResult>` via executor service.
  - Code editor typing remains responsive (60 FPS); UI never freezes during scans.
- **Main Thread Dispatch (`Platform.runLater`)**:
  - Upon task completion, results bind to UI controls:
  i. `renderTokens()`: Populates `TableView<Token>` and updates token search filter.
  ii. `renderAst()`: Constructs recursive `TreeView<SyntaxNode>` nodes.
  iii. `renderDiagnostics()`: Clears and repopulates `VBox` diagnostic cards.

Slide 14: Phase 5 — Diagnostics Cards & Interactive Caret Jump

From Diagnostic Data Model to Interactive UI Element:

```
// Inside WorkspaceController.createDiagnosticCard()
VBox card = new VBox(6);
card.getChildren().addAll(headerBadges, messageLabel, fixLabel);

card.setOnMouseClicked(ev -> {
    jumpToLocation(diag.line(), diag.column());
    highlightLine(diag.line(), diag.column());
});
```

- **Visual Hierarchy**: Severity Pill (`ERROR`/`WARNING`) + Error Code + Location Badge + Category Pill.
- **Coordinate Mapping Seam**:
  - Editor uses raw character offsets (`text.charAt(i)`).

Slide 15: Key Design Tradeoffs & Architectural Summary

Tradeoff	Selected Approach	Rationale & Alternative Rejected
Storage Model	100% In-Memory Pipeline	Rejected disk serialization/caching for maximum speed and simplicity (ADR 0001).
AST Representation	Lightweight `SyntaxNode` + typed records	Rejected heavy compiler frameworks (ANTLR/Eclipse AST) to eliminate bloat and maintain zero runtime baggage.
Pipeline Resilience	Fault-Tolerant Stage Containment	Never throws on parse error; converts failures to `ERR_INTERNAL_CHECK_FAILURE` diagnostics.
Diagnostic	Two-Tier Canonical	Internal parser recovery diagnostics cleanly yield to actionable canonical rules